```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("/content/city_day.csv")
df.head()
```

	City	Datetime	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene	Xylene	AQI	AQI_Bucket
0	Delhi	2015-01-01	153.3	241.7	182.9	33.0	81.3	38.5	1.87	64.5	83.6	18.93	20.81	8.32	204.5	Severe
1	Mumbai	2015-01-01	70.5	312.7	195.0	42.0	122.5	31.5	7.22	83.8	108.0	2.01	19.41	2.86	60.9	Satisfactory
2	Chennai	2015-01-01	174.1	275.4	56.2	68.8	230.9	28.5	8.56	60.8	43.9	19.07	10.19	9.63	486.5	Severe

Next steps: Generate code with df New interactive sheet

```
df = df.drop(['City', 'Date'], axis=1, errors='ignore')
```

```
df = df.fillna(df.mean(numeric_only=True))
df = df.dropna()
```

```
def simplify_category(x):
    if x in ['Good', 'Satisfactory']:
        return 0   # Low pollution
    else:
        return 1   # High pollution

df['AQI_Bucket'] = df['AQI_Bucket'].apply(simplify_category)
```

```
X = df.drop(['AQI_Bucket', 'AQI'], axis=1)
y = df['AQI_Bucket']
```

```
X = X.select_dtypes(include=[np.number])
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=3000)
lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)
```

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(max_depth=8, random_state=42)
dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)
```

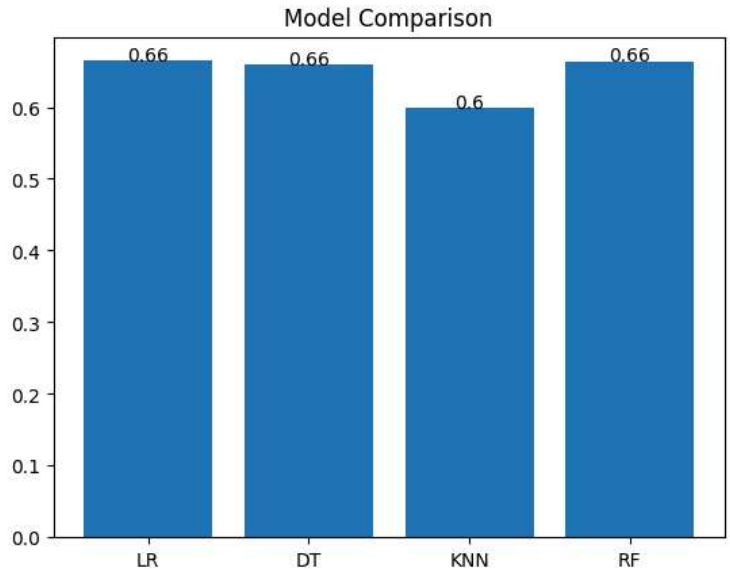
```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)

knn_pred = knn.predict(X_test)
```

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=15,
    random_state=42
)

rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
```

```
from sklearn.metrics import accuracy_score

print("Logistic Regression:", accuracy_score(y_test, lr_pred))
print("Decision Tree:", accuracy_score(y_test, dt_pred))
print("KNN:", accuracy_score(y_test, knn_pred))
print("Random Forest:", accuracy_score(y_test, rf_pred))
```

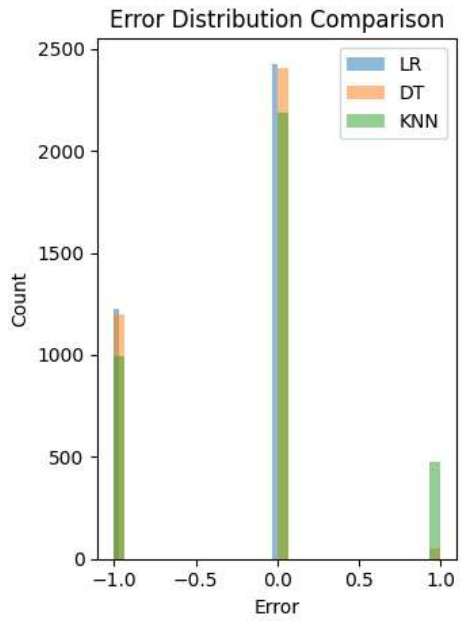
```
Logistic Regression: 0.664659184232138
Decision Tree: 0.6589104845332603
KNN: 0.598412263892691
Random Forest: 0.664111689022721
```

```
models = ['LR', 'DT', 'KNN', 'RF']
accuracies = [
    accuracy_score(y_test, lr_pred),
    accuracy_score(y_test, dt_pred),
    accuracy_score(y_test, knn_pred),
    accuracy_score(y_test, rf_pred)
]

plt.figure()
plt.bar(models, accuracies)

for i, v in enumerate(accuracies):
    plt.text(i, v, str(round(v, 2)), ha='center')

plt.title("Model Comparison")
plt.show()
```

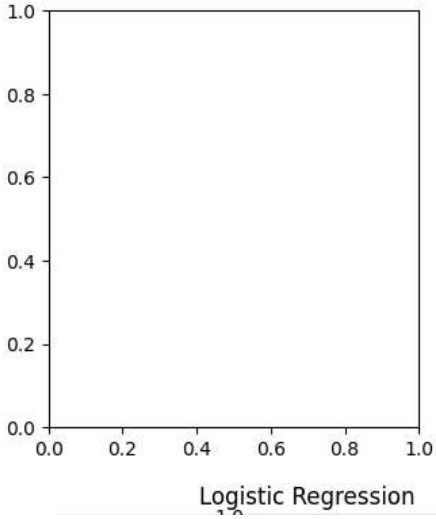



```
# RIGHT GRAPH
plt.subplot(1,2,2)

# Errors
lr_error = y_test - lr_pred
dt_error = y_test - dt_pred
knn_error = y_test - knn_pred

plt.hist(lr_error, bins=30, alpha=0.5, label='LR')
plt.hist(dt_error, bins=30, alpha=0.5, label='DT')
plt.hist(knn_error, bins=30, alpha=0.5, label='KNN')
```

```
plt.xlabel("Error")
plt.ylabel("Count")
plt.title("Error Distribution Comparison")
plt.legend()

plt.tight_layout()
plt.show()
```




```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

plt.figure(figsize=(12,4))

# Logistic Regression
plt.subplot(1,3,1)
cm = confusion_matrix(y_test, lr_pred)
ConfusionMatrixDisplay(cm).plot()
plt.title("Logistic Regression")

# Decision Tree
plt.subplot(1,3,2)
cm = confusion_matrix(y_test, dt_pred)
ConfusionMatrixDisplay(cm).plot()
plt.title("Decision Tree")

# KNN
plt.subplot(1,3,3)
cm = confusion_matrix(y_test, knn_pred)
ConfusionMatrixDisplay(cm).plot()
plt.title("KNN")

plt.tight_layout()
plt.show()
```


1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Logistic Regression